

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Fundamentals

Runtime CRI

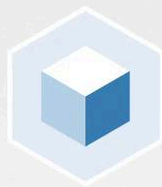
In this article, we explore the Container Runtime Interface (CRI) and its critical role in the container ecosystem. The CRI is a fundamental component that enables Kubernetes to support a variety of container runtimes without modifying its core codebase.

Background and Evolution

Initially, Docker became the most popular solution for container operations due to its simplicity. When Kubernetes was first introduced, it was designed to orchestrate Docker containers exclusively. However, as Kubernetes grew in popularity, new container technologies such as Rocket and ContainerD emerged that required integration with Kubernetes. To address this need, Kubernetes introduced the Container Runtime Interface (CRI).

CRI defines a plugin interface that any container vendor can implement if they adhere to the Open Container Initiative (OCI) standards. This design involves a gRPC API used by the Kubernetes Kubelet to manage container images, containers, and networking. By implementing the CRI API, container runtimes can operate independently of Kubernetes, allowing system architects the flexibility to choose the optimal runtime for their environment.

Container Runtime Interface (CRI)



The CRI defines the gRPC Protocol that the Kubernetes Kubelet uses to interact with container runtimes



Images



Container



Networking

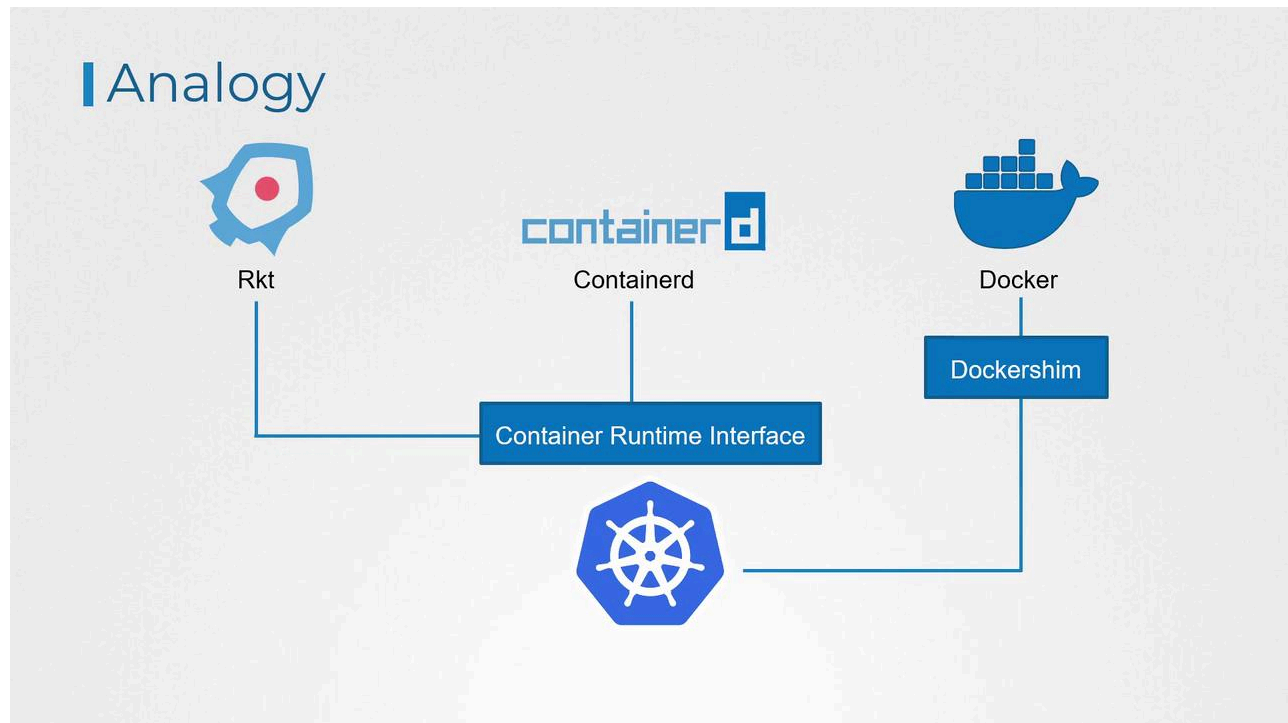


Note

For more details on container orchestration and Kubernetes, consider reviewing the [Kubernetes Documentation](#).

Docker and the Introduction of Docker Shim

Docker's widespread adoption in the container ecosystem meant that even as CRI was introduced, the Kubernetes community continued to support Docker. To maintain compatibility, Kubernetes implemented a temporary solution known as Docker Shim. This intermediary layer allowed Docker to communicate with Kubernetes without directly using the CRI, ensuring that existing Docker-based workflows continued to function seamlessly.

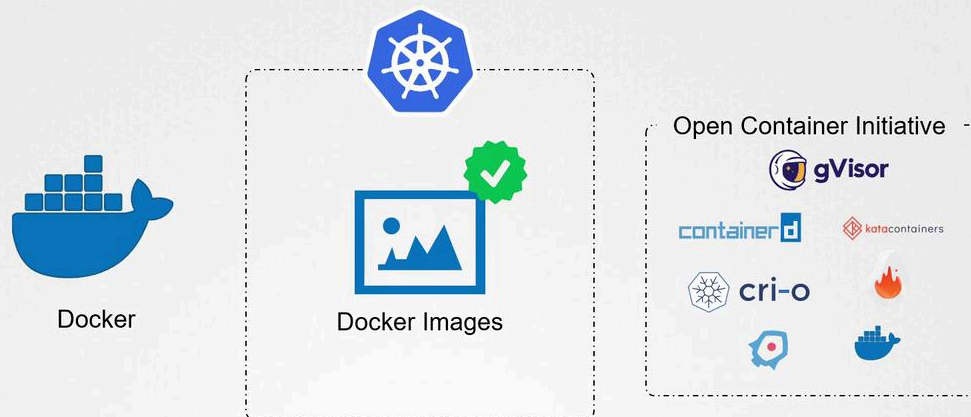


Docker Shim was maintained purely for backward compatibility. As Kubernetes evolved toward a more container runtime-agnostic approach, the reliance on Docker Shim was deprecated. In Kubernetes version 1.24, Docker Shim was officially removed, and native support for Docker was dropped. However, Docker images remain compatible because they adhere to the OCI standard, which means they can be used with other container runtimes like ContainerD.

Embracing Container-Native CRI Support

Users are now encouraged to adopt container runtimes that natively support the CRI. This shift not only enhances compatibility and standardization across container runtimes but also helps mitigate vendor lock-in. By supporting multiple container runtimes, Kubernetes enables organizations to choose the most appropriate solution for their infrastructure needs.

Docker Removal



Additional Resources

Explore further insights into container runtimes and Kubernetes best practices by visiting the [Kubernetes Basics](#) and other related resources.

Conclusion

The Container Runtime Interface has paved the way for enhanced compatibility and flexibility in the container ecosystem. By enabling a diverse set of container runtimes, CRI ensures that Kubernetes users can avoid vendor-specific limitations and foster innovation in container management.

That concludes this article on the Container Runtime Interface. Thank you for reading, and we'll see you in the next one.



Beta



Kubernetes and Cloud Native Associate - KCNA

Previous

◀ **Kubernetes Architecture**

Next

Docker vs ContainerD ▶